

Документация на проект

Тема №5: Футболно първенство

Обектно-ориентирано програмиране

C++ конзолно приложение

Проектът реализира симулация на футболно първенство със сезони, отбори, играчи, мачове, класиране, голмайстори, трансфери и запис/зареждане от текстови файлове.

1. Анализ на задачата и подход за решение

Задачата изисква да се разработи система за симулация на футболно първенство. Основните обекти в предметната област са играч, отбор, стартов състав, мач, сезон/първенство и колекция от сезони. Поради това решението е разделено на отделни класове, като всеки клас отговаря за конкретна част от логиката.

Избраният подход е обектно-ориентиран. Използвани са капсулация, наследяване, полиморфизъм, динамична памет и изключения. Класът Team е абстрактен базов клас, а конкретните видове отбори са реализирани чрез наследници: AttackTeam, DefenceTeam и BalanceTeam. Това позволява общото поведение на отборите да бъде на едно място, а различните правила за състав и покупка на играчи да се реализират чрез виртуални функции.

Основни стъпки на решението

- Дефиниране на клас Player за съхраняване на информация и статистика за футболист.
- Дефиниране на базов клас Team и наследници за различните типове отбори.
- Реализиране на Lineup за избор и проверка на стартов състав.
- Реализиране на Match за изиграване на мач, въвеждане/генериране на резултат и обновяване на статистики.
- Реализиране на Championship за един сезон, генериране на програма и класиране.
- Реализиране на ChampionshipCollection за история от първенства.
- Добавяне на интерактивно меню и работа с текстови файлове.

Избрани алгоритми и решения

За генериране на програмата се използва round-robin алгоритъм. При четен брой отбори всеки кръг съдържа $\text{teamCount} / 2$ мача. Първата половина на сезона съдържа $\text{teamCount} - 1$ кръга, а втората половина обръща домакинството. Така всеки отбор играе срещу всеки друг два пъти — веднъж като домакин и веднъж като гост.

Формули:

- $\text{брой мачове} = \text{брой отбори} * (\text{брой отбори} - 1)$
- $\text{брой кръгове} = 2 * (\text{брой отбори} - 1)$
- $\text{мачове в един кръг} = \text{брой отбори} / 2$

Класирането се определя чрез сортиране по точки, а при равенство — по голова разлика.

Голмайсторът се избира измежду играчите с поне 7 мача. При равен брой голове печели играчът с по-малко изиграни мачове. Ако и това е равно, се извеждат всички равни голмайстори.

2. Кратко описание на класовете

Архитектурата е изградена около няколко основни класа. Таблицата обобщава тяхната роля.

Клас	Роля	Основна употреба
Player	Описва един футболист.	Съхранява позиция, номер, заплата, мачове и голове.
Team	Абстрактен базов клас за отбор.	Държи играчи, бюджет и статистика; дефинира общи операции.
AttackTeam	Атакуващ отбор.	Специални правила: минимум 8 нападатели и условие за голове при покупка на нападател.
DefenceTeam	Дефанзивен отбор.	Специални правила: повече защитници и халфове.
BalanceTeam	Балансиран отбор.	Равномерно разпределение по позиции.
Lineup	Стартов състав.	Избира и проверява 11 играчи за мач.
Match	Един мач.	Избира състави, резултат, голмайстори и обновява статистики.
Championship	Един сезон.	Държи отбори, мачове, програма и класиране.
ChampionshipCollection	История от сезони.	Добавя, търси, записва и зарежда първенства.

2.1. Клас Player

Класът Player описва един футболист. Член-данните са име, номер, позиция, заплата, трансферна сума, брой изиграни мачове и брой голове. Името се пази динамично. Позицията се представя чрез изброим тип Position.

Основни член-функции:

- конструктори, копиращ конструктор, оператор= и деструктор
- сетъри и гетъри за данните
- print() за извеждане на информация
- regulateSalary() за промяна на заплата според представянето
- saveToFile() и loadFromFile() за текстов файл

2.2. Клас Team

Team е абстрактен базов клас за всички отбори. Съдържа име на отбор, треньор, стадион, бюджет, масив от 20 играчи и статистики: мачове, победи, равенства, загуби, точки, вкарани и получени голове.

Основни член-функции:

- addPlayer() и removePlayer() за управление на играчи
- canBuyPlayer(), isValidTeam(), clone() и getType() като виртуални функции
- updateStatistics() за статистика след мач
- swapPlayersWith() и canSwapPlayersWith() за размяна
- saveToFile() и loadFromFile() за отбор и играчи

2.3. Клас AttackTeam

AttackTeam наследява Team и реализира правилата за атакуващ отбор. Валидният състав има точно 2 вратари, минимум 8 нападатели и минимум по 2 защитници, халфове и крила.

Основни член-функции:

- isValidTeam() проверява позиционните ограничения
- canBuyPlayer() добавя условие нападателят да има поне 6 гола
- clone() позволява полиморфно копиране
- getType() връща AttackTeam за save/load

2.4. Клас DefenceTeam

DefenceTeam наследява Team и описва дефанзивен отбор. Той трябва да има точно 2 вратари, минимум 6 защитници, минимум 6 халфове и минимум по 2 крила и нападатели.

Основни член-функции:

- isValidTeam() проверява дефанзивните ограничения
- canBuyPlayer() използва общите проверки за бюджет и лимити
- clone() и getType() се използват при копиране и зареждане

2.5. Клас BalanceTeam

BalanceTeam наследява Team и описва балансиран отбор. Той трябва да има точно 2 вратари и минимум по 4 защитници, халфове, крила и нападатели.

Основни член-функции:

- isValidTeam() проверява балансираното разпределение
- canBuyPlayer() проверява общите ограничения
- clone() и getType() позволяват работа чрез Team*

2.6. Клас Lineup

Lineup описва стартовите 11 играча на конкретен отбор. Съдържа указател към Team, масив от Player* и текущ размер. Използва се в Match.

Основни член-функции:

- choosePlayersFromConsole() за ръчен избор
- chooseAutomatically() за автоматичен валиден състав
- isValid() проверява 11 играчи, точно 1 вратар и минимумите по позиции
- tryIncludePlayersWithLessThan3Matches() включва играчи с под 3 мача в последните кръгове
- markMatchPlayed() увеличава мачовете на избраните играчи

2.7. Клас Match

Match описва един мач между домакин и гост. Съдържа указатели към двата отбора, два Lineup обекта, голове, кръг, общ брой кръгове, списък с голмайстори и флаг дали е изигран.

Основни член-функции:

- playMatch() за ръчен мач
- playMatchAutomatically() за автоматичен мач

- `enterGoalScorers()` проверява дали голмайсторът е в състава
- `addAutomaticGoalScorers()` избира голмайстори при автоматичен мач
- `print()` показва резултат и голмайстори

2.8. Клас `Championship`

`Championship` описва един сезон. Съдържа година, масив от `Team*`, масив от `Match`, текущ кръг и флаг `finished`.

Основни член-функции:

- `addTeam()` добавя отбор чрез `clone()`
- `generateSchedule()` генерира round-robin програма
- `playRound()` изиграва текущ кръг ръчно или автоматично
- `printStandings()` извежда класиране
- `printFinalResults()` извежда шампион, втори, трети и голмайстор
- `saveToFile()` и `loadFromFile()` записват/зареждат сезон

2.9. Клас `ChampionshipCollection`

`ChampionshipCollection` съдържа динамичен масив от първенства. Използва се за история от сезони и за работа с менюто.

Основни член-функции:

- `addChampionship()` добавя сезон с уникална година
- `startNewSeason()` автоматично избира следваща година
- `getChampionshipByYear()` търси сезон
- `printAllChampionships()` извежда списък
- `saveToFile()` и `loadFromFile()` пазят всички сезони

3. Начин на използване на програмата

Програмата се управлява чрез интерактивно конзолно меню. Основните операции са:

- `Start new season` — създава нов сезон с автоматична година.
- `Add championship with custom year` — добавя сезон с избрана година.
- `Add team to season` — добавя отбор и неговите играчи.
- `Show teams in season` — извежда отборите в даден сезон.
- `Generate schedule` — генерира програмата за мачове.
- `Play one round` — изиграва текущия кръг ръчно или автоматично.
- `Play all rounds` — изиграва всички оставащи кръгове.
- `Show standings` — показва класиране.
- `Swap players between teams` — разменя играчи при валидни състави след размяната.
- `Show all championships` — показва сезоните в колекцията.
- `Save championships to file` — записва всички сезони в текстов файл.
- `Load championships from file` — зарежда сезони от текстов файл.
- `Show final results` — показва шампион, вицешампион, трети отбор и голмайстор.
- `Regulate salaries for team` — регулира заплатите на играчите в избран отбор според изиграните мачове, головете и позицията им.

4. Записване и зареждане от файл

Проектът използва текстови файлове. Записването става чрез оператор <<, а зареждането чрез >> и getline(). Форматът е фиксиран: първо се записва броят на първенствата, след това за всяко първенство се записват година, текущ кръг, finished, брой отбори, тип на отбор, данни за отбор и данни за играчите.

Записват се:

- всички първенства
- година, текущ кръг и finished
- типове отбори: AttackTeam, DefenceTeam, BalanceTeam
- статистика на отборите
- играчи и тяхната статистика

Мачовете не се записват директно, защото съдържат указатели към отбори. След зареждане програмата може отново да генерира програмата за мачове. Това опростява формата и избягва проблеми с възстановяване на указатели.

5. Подготвени тестови файлове

Към проекта са добавени седем текстови файла за демонстрация и проверка:

- **01_basic_4_teams.txt** — Базово първенство с 4 отбора. Подходящо за зареждане, генериране на програма и автоматично изиграване на кръг.
- **02_standings_and_scorers.txt** — Първенство с предварителна статистика за класиране и играчи.
- **03_swap_players.txt** — Файл за демонстрация на размяна на играчи между два отбора.
- **04_multiple_seasons.txt** — Колекция с два сезона — един завършен и един незавършен.
- **05_last_round_rule.txt** — Файл за демонстрация на правилото за играчи с по-малко от 3 мача в последните кръгове.
- **06_top_scorer_test.txt** — Файл за демонстрация на финални резултати и избор на голмайстор при равни голове и различен брой мачове.
- **07_salary_test.txt** — Файл за демонстрация и проверка на функцията за регулиране на заплатите на играчите според изиграни мачове, отбелязани голове и позиция.

6. Обработка на грешки

За невалидни операции се използват изключения от <stdexcept>. Това позволява програмата да спре конкретната грешна операция, без да се прекратява цялото приложение.

- невалиден брой отбори
- невалидна позиция
- дублирано име или номер на играч
- повече от 20 играчи
- невалиден стартов състав
- невалидна размяна
- дублирана година на сезон
- невъзможно отваряне на файл

7. Работа с динамична памет

Проектът използва динамична памет за символни низове, масиви от отбори, мачове и колекция от първенства. За класовете, които управляват динамична памет, са реализирани голяма четворка/подходящи методи: конструктор, копиращ конструктор, оператор за присвояване и деструктор. Полиморфното копиране на отборите се извършва чрез виртуалната функция `clone()`.

8. Идеи за бъдещи подобрения

- Отделна класация за асистенции.
- Контузии и наказания.
- Трансферни прозорци.

9. Заключение

Проектът реализира цялостна обектно-ориентирана симулация на футболно първенство. Решението включва работа с наследяване, полиморфизъм, динамична памет, файлове и интерактивно меню. Системата позволява създаване и управление на сезони, отбори и играчи, провеждане на мачове, генериране на класиране, определяне на шампион и голмайстор, размяна на играчи и работа с подготвени тестови файлове.